Dynamic Encryption/Decryption Document for External Clients

Dynamic Encryption:

Workflow:

```
Input JSON Data
    ↓
[Step 1] Generate a random key (Session Key)
    ↓
[Step 2] encrypt JSON Data with Session Key → encryptedData
    ↓
[Step 3] Sign the encryptedData with your external system's private key → signData
    ↓
[Step 4] Get GepNIC public key thumbprint → keyId
    ↓
[Step 5] encrypt the Session Key with GepNIC public key → encryptKey
   ↓
uniqueUserId, encryptData, signData, encryptKey, keyId
```

uniqueUserId	Unique ID shared by GepNIC with the external system
encryptData	The actual JSON (input data) encrypted using the dynamically generated Session Key (256-bit AES symmetric key)
encryptKey	The Session Key (256-bit AES symmetric key) encrypted using GepNIC's public key (gepnic.pem)
signData	The encryptData value digitally signed using external system's private key (ExtSysprivate.pfx)
keyId	The thumbprint (SHA-1) of the GepNIC public key (gepnic.pem)

1: Generate Dynamic Session Key

Generate a random 256-bit AES Session Key for this request:

```
 public static String generateSessionKey() {
    KeyGenerator keyGenerator;
    try {
        keyGenerator = KeyGenerator.getInstance("AES");
        SecureRandom secureRandom = new SecureRandom();
        int keyBitSize = 256;
        keyGenerator.init(keyBitSize, secureRandom);

        SecretKey secretKey = keyGenerator.generateKey();
        return Base64.getEncoder().encodeToString(secretKey.getEncoded());
    } catch (NoSuchAlgorithmException e) {
        return "Error";
    }
}
```

2: Encrypt JSON Data with Session Key

Encrypt Input JSON Data using the Session Key

```
 public static String encryptData(String data, String sessionKeyBase64) throws
Exception {
    byte[] keyBytes = Base64.getDecoder().decode(sessionKeyBase64);
    SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "AES");

    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);
    byte[] encrypted = cipher.doFinal(data.getBytes("UTF-8"));
```

```
    return Base64.getEncoder().encodeToString(encrypted);
}
```

3: Sign the Encrypted Data

Sign the encrypted data using external system's private key (SHA-512 with RSA):

```
public static String signData(String encryptedData, String keystoreFilePath,
        String password, String alias, String extensionType) throws Exception {

    PrivateKey privateKey;
    if (extensionType.equalsIgnoreCase("jks")) {
        privateKey = privateKeyFromJKSFile(keystoreFilePath, password, alias);
    } else if (extensionType.equalsIgnoreCase("pfx")) {
        privateKey = privateKeyFromPFXCert(keystoreFilePath, password);
    } else {
        throw new IllegalArgumentException("Unsupported certificate type: " +
extensionType);
    }

    Signature signature = Signature.getInstance("SHA512withRSA");
    signature.initSign(privateKey);
    signature.update(encryptedData.getBytes("UTF-8"));
    byte[] signArray = signature.sign();

    return Base64.getEncoder().encodeToString(signArray);
}
```

4: Load GepNIC Public Key and Generate Thumbprint

Take GepNIC certificate file and extract two things the public key and the certificate thumbprint.

```
public static Map<String, Object> getPublicKeyAndThumbprint(String certificatePath)
throws Exception {
    Map<String, Object> result = new HashMap<>();

    PublicKey publicKey = publicKeyFromPEMFile(certificatePath);
    String thumbprint = generateThumbprint(certificatePath);

    result.put("publicKey", publicKey);
    result.put("thumbprint", thumbprint);

    return result;
}

public static PublicKey publicKeyFromPEMFile(String certificatePath) throws Exception
{
    InputStream is = new FileInputStream(certificatePath);
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    X509Certificate cert = (X509Certificate) cf.generateCertificate(is);
    return cert.getPublicKey();
}

public static String generateThumbprint(String certificatePath) throws Exception {
    X509Certificate certificate = getCertObject(certificatePath);
    MessageDigest md = MessageDigest.getInstance("SHA-1");
```

```
    md.update(certificate.getEncoded());
    // Return hex string in lowercase (matches server implementation)
    return DatatypeConverter.printHexBinary(md.digest()).toLowerCase();
}

public static X509Certificate getCertObject(String filePath) throws Exception {
    try (FileInputStream is = new FileInputStream(filePath)) {
        CertificateFactory cf = CertificateFactory.getInstance("X.509");
        return (X509Certificate) cf.generateCertificate(is);
    }
}
```

5: Encrypt Session Key with GepNIC Public Key

Encrypt the Session Key using GepNIC's RSA public key:

```
 public static String encryptSessionKey(String sessionKeyBase64, PublicKey publicKey) throws Exception {
    Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
    cipher.init(Cipher.ENCRYPT_MODE, publicKey);
    byte[] encrypted = cipher.doFinal(sessionKeyBase64.getBytes("UTF-8"));
    return Base64.getEncoder().encodeToString(encrypted);
}
```

Dynamic Decryption:

Json Response:

```
{
  "uniqueUserId": "user_id",
  "encryptData": "base64_encrypted_response_data",
  "signData": "base64_digital_signature",
  "encryptKey": "base64_encrypted_Session Key ",
  "keyId": "base64_thumbprint"
}
```

Workflow:

GepNIC's `Response (uniqueUserId ,encryptData, signData, encryptKey, keyId)`
↓
[Step 1] Verify signature with GepNIC's public key
↓
[Step 2] Verify keyId matches GepNIC's certificate
↓
[Step 3] Decrypt encryptKey with external system's private key → **Session Key**
↓
[Step 4] Decrypt encryptData with Session Key → JSON response
↓
`Read the decrypted JSON data`

When receive a response from GepNIC:

1. Verify the digital signature using GepNIC's public key
2. Decrypt the encrypted Session Key using external system's private key
3. Using the Session Key, decrypt the JSON response data

Response Decryption Implementation

1: Verify Signature

Verify the digital signature using GepNIC's public key (SHA-512 with RSA):

```
public static boolean verifySignature(String data, String signData, PublicKey publicKey) throws Exception {

    Signature signature = Signature.getInstance("SHA512withRSA");
```

```
    signature.initVerify(publicKey);
    signature.update(data.getBytes("UTF-8"));
    return signature.verify(Base64.getDecoder().decode(signData));
}
```

2: Verify KeyId

Verify that the keyId matches GepNIC's certificate thumbprint:

```
 public static String generateThumbprint(String certificatePath) throws Exception {
    X509Certificate certificate = getCertObject(certificatePath);
    MessageDigest md = MessageDigest.getInstance("SHA-1");
    md.update(certificate.getEncoded());
    // Return hex string in lowercase (matches server implementation)
    return DatatypeConverter.printHexBinary(md.digest()).toLowerCase();
}
```

3: Decrypt Session Key

Decrypt the Session Key using external system's private key:

```
 public static String decryptSessionKey(String encryptedKey, PrivateKey privateKey)
throws Exception {
    Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
    cipher.init(Cipher.DECRYPT_MODE, privateKey);
    byte[] decrypted = cipher.doFinal(Base64.getDecoder().decode(encryptedKey));
    return new String(decrypted, "UTF-8");
}
```

4: Decrypt Data

Decrypt the response data using the Session Key:

```
 public static String decryptData(String encryptedData, String sessionKeyBase64)
throws Exception {
    byte[] keyBytes = Base64.getDecoder().decode(sessionKeyBase64);
    SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "AES");
```

```
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
    cipher.init(Cipher.DECRYPT_MODE, secretKey);
    byte[] decrypted = cipher.doFinal(Base64.getDecoder().decode(encryptedData));

    return new String(decrypted, "UTF-8");
}
```

Data Encryption

1. Algorithm: AES-256-ECB
2. Padding: PKCS5Padding
3. Key Size: 256 bits (32 bytes)
4. Session Key: Randomly generated per request
5. Encoding: Base64

Session Key Encryption

1. Algorithm: RSA-2048-ECB
2. Padding: PKCS1Padding
3. Key: GepNIC public key (for requests)
4. Encoding: Base64

Digital Signature

1. Algorithm: RSA with SHA512
2. Certificate: external system's private key
3. Format: Base64 encoded signature

KeyId (Thumbprint)

1. Algorithm: SHA-1
2. Input: GepNIC public key certificate
3. Purpose: Verify which GepNIC certificate was used
4. Encoding: Hex string (lowercase)

Certificate Requirements

External system's Certificate (DSC)

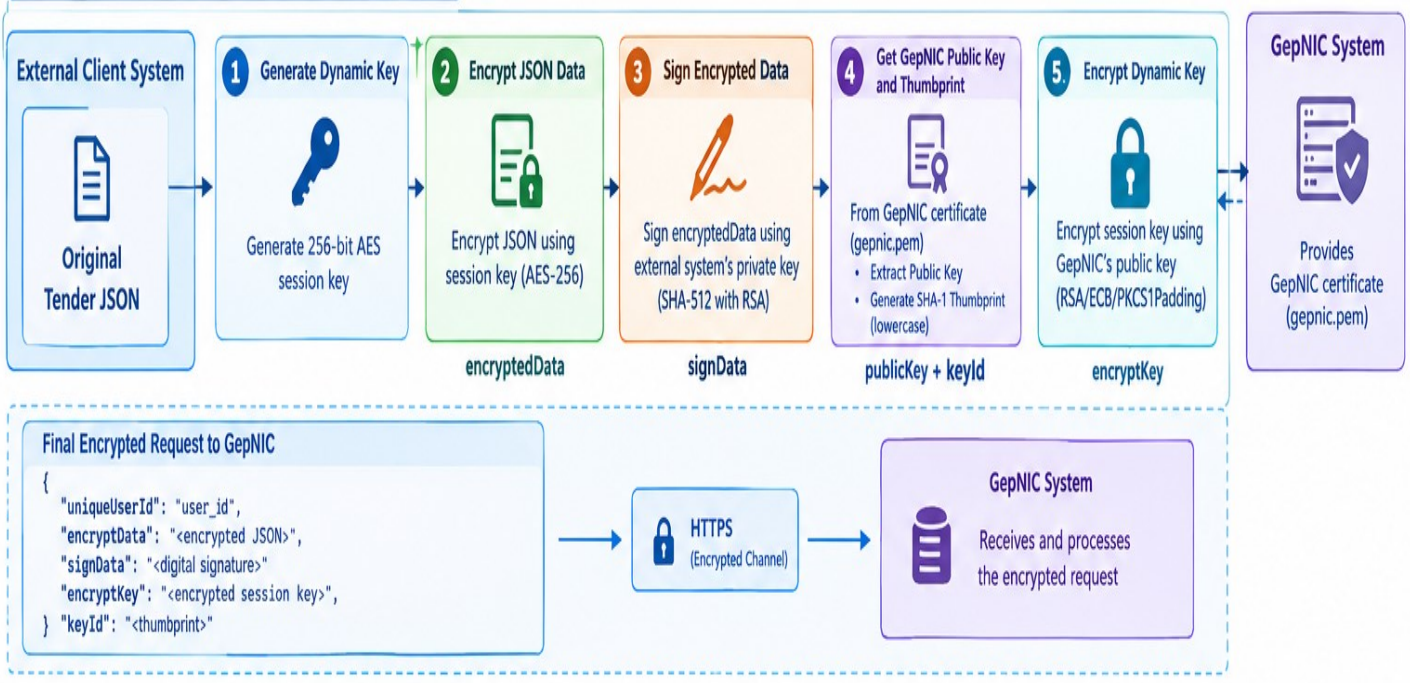
1. Format: PKCS#12 (.pfx) or JKS (.jks) keystore
2. Key Size: Minimum 2048-bit RSA
3. Algorithm: RSA with SHA-512
4. Validity: Must be valid at time of request
5. Registration: Must be registered with GepNIC

GepNIC Public Key

1. Obtain GepNIC's public certificate (gepnic.pem)
2. Store securely in your application
3. Use for encrypting Session Key and verifying signatures

A. Dynamic Encryption – Request Flow

(External Client System → GepNIC System)



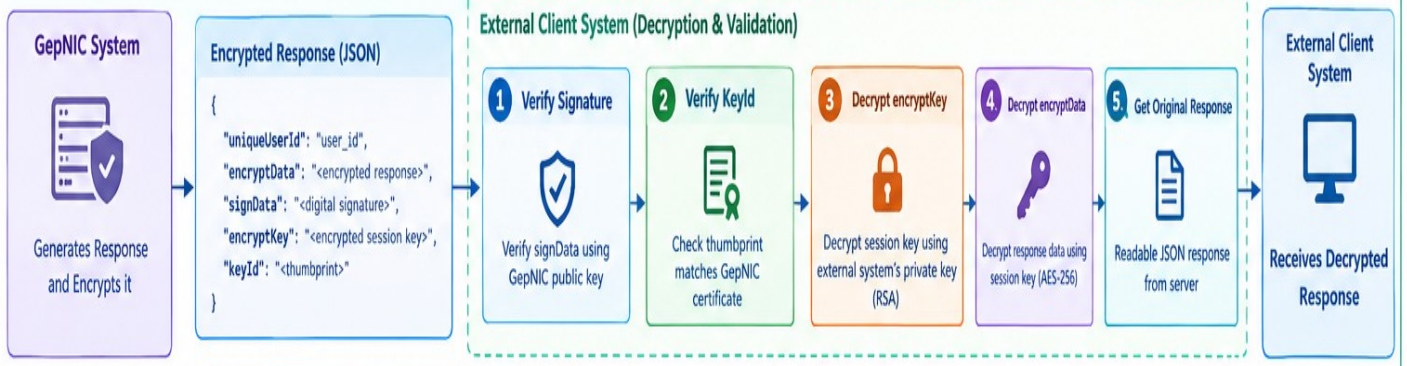
Final Encrypted Request to GepNIC

```
{
  "uniqueUserId": "user_id",
  "encryptData": "<encrypted JSON>",
  "signData": "<digital signature>"
  "encryptKey": "<encrypted session key>",
} "keyId": "<thumbprint>"
```

HTTPS (Encrypted Channel) → GepNIC System: Receives and processes the encrypted request

B. Dynamic Decryption – Response Flow

(GepNIC System → External Client System)



Final Response (Decrypted)

```
{
  "responseCode": 1,
  "responseMessage": {\"responseCode\":101,\"responseMessage\":\"Inserted Successfully\"}"
}
```

Key Points

- Each request uses a unique 256-bit AES session key.
- JSON is encrypted with AES, session key is encrypted with GepNIC's RSA key.
- Digital signature ensures authenticity and integrity.
- keyId (SHA-1 thumbprint) identifies the GepNIC certificate.
- Response follows the reverse process.

Request Parameters (as per document)

uniqueUserId	encryptData	encryptKey	signData	keyId
Unique ID shared by GepNIC with the external system	JSON encrypted using dynamic AES key (256-bit)	Dynamic key encrypted using GepNIC's public key	encryptData signed using external system's private key	SHA-1 thumbprint of GepNIC public key (lowercase hex)